

Campus Connect - Code Summary

Qt client + Boost.Asio server + JSON over TCP

Build and Entry Points

CMakeLists.txt: builds app.exe, server.exe, final_tests_client, and final_tests_server. It links Qt for the GUI/client networking, Boost.Asio/Windows socket libraries for the server, and GoogleTest/GoogleMock for tests.

src/main.cpp: starts QApplication, creates TcpNetworkClient, injects it into ChatController through INetworkClient, opens LoginWindow, and runs the Qt event loop.

server.cpp: starts the CLI server by creating boost::asio::io_context, constructing TcpServer on port 12345, and running the asynchronous event loop.

Client GUI Layer - src/gui

LoginWindow: collects username, server IP, and port, then calls ChatController::connectUser. On success it opens ChatWindow. LoginWindow.ui stores the Qt Designer layout.

ChatWindow: main public chat screen. It displays public messages, online users, typing status, and controls for text, voice, attachments, camera media, groups, and private chats.

PrivateWindow: private chat dialog for one user. It sends private messages and attachments through ChatController and renders private conversation history.

GroupWindow: group chat screen. It creates/joins groups, sends group messages, and displays group-specific messages and typing status.

MessageRenderer: converts Message objects and attachment markers into HTML for the chat displays, including text, voice, image, video, file links, group messages, and typing indicators.

Client Business Logic - src/logic

ChatController: central client controller. It validates actions, builds Message objects, serializes JSON, sends through INetworkClient, receives incoming callbacks, updates ChatState, and manages attachment metadata/chunks.

ChatState: stores current user, online users, public/private/group messages, joined groups, and typing-state data. GUI reads state instead of owning chat logic.

Message: shared protocol model. It defines message types such as connect, public_message, private_message, group_message, typing, attachment_start, and attachment_end. It converts between C++ objects and Qt JSON.

InputValidator: checks username and message validity before ChatController allows connect/send operations.

Campus Connect - Code Summary (continued)

Client Networking - src/network

INetworkClient: abstract interface used by ChatController. This keeps networking separate from business logic and lets tests inject a mock client.

TcpNetworkClient: real Qt TCP client. It connects to the selected host/port, sends framed JSON text and binary attachment chunks, reads frames asynchronously using

QTcpSocket::readyRead, and dispatches payloads to callbacks.

MockNetworkClient: fake/mock network implementation used for isolated controller tests or local non-socket scenarios.

Server Layer - src/server

TcpServer: Boost.Asio acceptor. It asynchronously accepts incoming sockets and creates one ClientSession per connected client while sharing one ServerController.

ClientSession: handles one client connection. It reads a 4-byte big-endian frame length, then a payload. Frame type 0 is JSON text; frame type 1 is binary data. It forwards data to ServerController and can send text/binary frames back.

ServerController: server routing/business logic. It registers/removes clients, handles connect/public/private/group/typing/attachment messages, broadcasts public messages, routes private messages, routes group messages, and updates user lists.

ServerState: server-side storage for users, sessions, groups, and membership data. It keeps routing state separate from Boost.Asio socket code.

Message Flow and Attachments

Flow: GUI input -> ChatController validation -> Message object -> Message::toJson ->

TcpNetworkClient frame -> TCP -> ClientSession -> ServerController routing -> receiving clients -> Message::fromJson -> ChatState update -> GUI refresh.

Attachments use attachment_start metadata, binary chunk frames identified by fileId, and attachment_end. The receiver saves chunks and renders the completed file as a voice/image/video/file link.

Tests - final_tests

Client tests: test_ChatController.cpp uses GoogleMock MockNetworkClient to verify validation and sending behavior without real sockets.

Server tests: test_ChatState.cpp checks joining groups, avoiding duplicate groups, storing group messages correctly, accumulating public messages, and default group membership.

Forwarding headers in final_tests/SERVER ensure tests use the real ChatState and Message classes instead of stale duplicate definitions.

Design Point

The code follows the required separation: GUI displays/collects input, logic classes enforce rules/state, networking classes handle sockets, server classes separate sessions from routing/state, and JSON is the shared protocol.